

Python

Module 2

(Collections, functions and Modules)

Accessing List

Theory:

- Understanding how to create and access elements in a list.

Definition

A **list** is an ordered and mutable collection of elements in Python that allows storing multiple values in a single variable.

You access list elements using **square brackets []** with an index.

- Indexing in lists (positive and negative indexing).

Positive Indexing

- Starts from **0**
- Moves left to right

```
numbers = [10, 20, 30, 40, 50]
```

```
# Indexes: 0  1  2  3  4
```

```
print(numbers[1]) # 20
```

```
print(numbers[4]) # 50
```

Negative Indexing

- Starts from **-1**
- Moves right to left

```
# Negative indexes: -5 -4 -3 -2 -1
```

```
print(numbers[-1]) # 50
```

```
print(numbers[-3]) # 30
```

[EXTRA PRACTICE QUESTION\accesseliment.py](#)

- Slicing a list: accessing a range of elements.

Slicing syntax:

`list[start : end : step]`

- start → index to begin (included)
- end → index to stop (excluded)
- step → gap between elements (optional)

Basic slicing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4]) # [20, 30, 40]
print(numbers[:3]) # [10, 20, 30]
print(numbers[2:]) # [30, 40, 50]
```

Slicing with negative indexes

```
print(numbers[-4:-1]) # [20, 30, 40]
```

Using step

```
print(numbers[::-2]) # [10, 30, 50]
print(numbers[::-1]) # [50, 40, 30, 20, 10] (reverse list)
```

[EXTRA PRACTICE QUESTION\alicing.py](#)

2. List Operations

a) Concatenation (+)

Definition

Concatenation is the operation of joining two or more lists to create a new single list.

```
list1 = [1, 2, 3]

list2 = [4, 5, 6]

result = list1 + list2

print(result)
```

Output:

csharp

Copy code

```
[1, 2, 3, 4, 5, 6]
```

b) Repetition (*)

Definition

Repetition is used to repeat the elements of a list a specified number of times.

Example

```
numbers = [1, 2]
result = numbers * 3
print(result)
```

Output:

```
[1, 2, 1, 2, 1, 2]
```

c) Membership (in, not in)

Definition

Membership operation is used to check whether an element exists in a list or not.

Example

```
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)
print("mango" not in fruits)
```

Output:

```
True
True
```

2. List Methods

a) append()

b) insert()

c) remove()

d) pop()

Operation / Method	Purpose
Concatenation (+)	Joins two lists
Repetition (*)	Repeats list elements
Membership (in)	Checks element existence
append()	Adds element at end
insert()	Adds element at position
remove()	Deletes specific element
pop()	Removes and returns element

[EXTRA PRACTICE QUESTION\MULTIPAL DATA.PY](#)

3. Working with Lists

1. Iterating Over a List Using Loops

Definition

Iterating over a list means accessing each element of the list one by one using a loop, usually a for loop or a while loop.

Example (using for loop)

```
numbers = [10, 20, 30, 40]
```

```
for num in numbers:
    print(num)
```

Output:

```
10
20
30
40
```

Example (using while loop)

```
i = 0
while i < len(numbers):
    print(numbers[i])
    i += 1
```

[EXTRA PRACTICE QUESTION\for loop.py](#)

2. Sorting and Reversing a List

a) sort()

Definition

The `sort()` method arranges the elements of a list in ascending order by default and modifies the original list.

Example

```
marks = [75, 60, 90, 80]
marks.sort()
print(marks)
```

Output:

```
[60, 75, 80, 90]
```

b) `sorted()`

Definition

The `sorted()` function returns a new sorted list without changing the original list.

Example

```
marks = [75, 60, 90, 80]
new_list = sorted(marks)
print(new_list)
print(marks)
```

Output:

```
[60, 75, 80, 90]
[75, 60, 90, 80]
```

c) `reverse()`

Definition

The `reverse()` method reverses the order of elements in the list.

Example

```
numbers = [1, 2, 3, 4]
numbers.reverse()
print(numbers)
```

Output:

```
[4, 3, 2, 1]
```

3. Basic List Manipulations

a) Addition

Definition

Addition in lists means inserting new elements using methods like `append()` or `insert()`.

Example

```
fruits = ["apple", "banana"]
fruits.append("mango")
print(fruits)
```

b) Deletion

Definition

Deletion removes elements from a list using methods like `remove()`, `pop()`, or the `del` keyword.

Example

```
fruits.remove("banana")
print(fruits)
```

c) Updating

Definition

Updating a list means changing the value of an existing element using its index.

Example

```
fruits[0] = "orange"
print(fruits)
```

d) Slicing

Definition

Slicing extracts a portion of a list using a range of indexes.

Example

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

Insert [EXTRA PRACTICE QUESTION\insert.py](#)

- **Iteration:** Accessing elements one by one using loops

- **sort():** Sorts original list
- **sorted():** Returns new sorted list
- **reverse():** Reverses list order
- **Addition:** append(), insert()
- **Deletion:** remove(), pop(), del
- **Updating:** Change element using index
- **Slicing:** Access multiple elements

4. Tuple Theory:

- Introduction to tuples, immutability.

1. Tuple (Introduction & Immutability)

Definition

A **tuple** is an ordered collection of elements in Python that is **immutable**, meaning its elements cannot be changed after creation.

□ **Immutability** means:

- You **cannot add**
- You **cannot remove**
- You **cannot modify** elements of a tuple

```
colors = ("red", "green", "blue")
```

```
print(colors)
```

Trying to modify a tuple causes an error:

```
python
```

Copy code

```
colors[0] = "yellow" # Error
```

- Creating and accessing elements in a tuple.

2. Creating and Accessing Elements in a Tuple

Creating a Tuple

Definition

A tuple is created by placing elements inside parentheses () separated by commas.

Example

```
numbers = (10, 20, 30, 40)
single = (5,) # Single-element tuple
```

□ A comma is required for a single-element tuple.

[EXTRA PRACTICE QUESTION\tuple.py](#)

Accessing Elements in a Tuple

Definition

Elements in a tuple are accessed using **indexing**, similar to lists.

Example

```
numbers = (10, 20, 30, 40)
print(numbers[1])
print(numbers[-1])
```

Output:

```
20
40
```

Type conversion [EXTRA PRACTICE QUESTION\conversion.py](#)

- Basic operations with tuples: concatenation, repetition, membership.

3. Basic Operations with Tuples

a) Concatenation (+)

Definition

Concatenation joins two or more tuples to form a new tuple.

Example

```
tuple1 = (1, 2)
tuple2 = (3, 4)
result = tuple1 + tuple2
print(result)
```

Output:

(1, 2, 3, 4)

b) Repetition (*)

Definition

Repetition repeats the elements of a tuple a specified number of times.

Example

```
t = (1, 2)
print(t * 3)
```

Output:

(1, 2, 1, 2, 1, 2)

c) Membership (in, not in)

Definition

Membership operation checks whether an element exists in a tuple.

Example

```
fruits = ("apple", "banana", "cherry")
print("banana" in fruits)
print("mango" not in fruits)
```

Output:

True
True

Short Exam-Ready Summary

- **Tuple:** Ordered, immutable collection
- **Immutability:** Elements cannot be changed

- **Creation:** Using parentheses ()
- **Access:** Using indexing
- **Concatenation:** +
- **Repetition:** *
- **Membership:** in, not in

5. Accessing Tuples

1. Accessing Tuple Elements Using Indexing

Definition

Indexing in a tuple is the process of accessing individual elements using their position (index). Python supports **positive indexing** and **negative indexing** for tuples.

a) Positive Indexing

Definition

Positive indexing starts from 0 for the first element and increases by 1 for each next element.

Example

```
colors = ("red", "green", "blue", "yellow")
```

```
print(colors[0])  
print(colors[2])
```

Output:

```
red  
blue
```

b) Negative Indexing

Definition

Negative indexing starts from -1 for the last element and moves backward.

Example

```
print(colors[-1])  
print(colors[-3])
```

Output:

```
yellow  
green
```

2. Slicing a Tuple

Definition

Slicing is used to extract a range of elements from a tuple.
The slicing operation returns a **new tuple**.

Syntax

tuple[start : end : step]

- start → starting index (included)
 - end → ending index (excluded)
 - step → gap between elements (optional)
-

Examples

```
numbers = (10, 20, 30, 40, 50)
```

Basic slicing

```
print(numbers[1:4])
```

Output:

```
(20, 30, 40)
```

Slicing from beginning

```
print(numbers[:3])
```

Output:

```
(10, 20, 30)
```

Slicing till end

```
print(numbers[2:])
```

Output:

```
(30, 40, 50)
```

Slicing with negative indexes

```
print(numbers[-4:-1])
```

Output:

```
(20, 30, 40)
```

Using step

```
print(numbers[::-2])
```

Output:

(10, 30, 50)

Add Method [EXTRA PRACTICE QUESTION\addtuple.py](#)

Short Exam-Ready Summary

- Tuples support **positive and negative indexing**
- Positive indexing starts from 0
- Negative indexing starts from -1
- **Slicing** extracts multiple elements
- Slicing always returns a **new tuple**
- End index is **not included**

6. Dictionaries

1. Dictionary (Introduction & Key-Value Pairs)

Definition

A **dictionary** is an unordered, mutable collection of data in Python that stores elements in **key–value pairs**.

- **Keys** must be unique and immutable (e.g., strings, numbers).
- **Values** can be of any data type.

Example

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "marks": 85  
}  
print(student)
```

2. Accessing Dictionary Elements

Definition

Dictionary elements are accessed using their **keys**, not indexes.

Example

```
print(student["name"])  
print(student["marks"])
```

Output:

Ali
85

[EXTRA PRACTICE QUESTION\dict.py](#)

3. Adding Elements to a Dictionary

Definition

New elements are added by assigning a value to a new key.

Example

```
student["grade"] = "A"  
print(student)
```

4. Updating Dictionary Elements

Definition

Updating means changing the value of an existing key.

Example

```
student["marks"] = 90  
print(student)
```

5. Deleting Dictionary Elements

Definition

Dictionary elements can be deleted using `del` or `pop()`.

Example (using `del`)

```
del student["age"]  
print(student)
```

Example (using `pop()`)

```
student.pop("grade")  
print(student)
```

6. Dictionary Methods

a) keys()

Definition

The keys() method returns all the keys of a dictionary.

Example

```
print(student.keys())
```

Output:

```
dict_keys(['name', 'marks'])
```

b) values()

Definition

The values() method returns all the values of a dictionary.

Example

```
print(student.values())
```

Output:

```
dict_values(['Ali', 90])
```

c) items()

Definition

The items() method returns all key–value pairs as tuples.

Example

```
print(student.items())
```

Output:

```
dict_items([('name', 'Ali'), ('marks', 90)])
```

[EXTRA PRACTICE QUESTION\dictoprations.py](#)

- **Dictionary:** Stores data in key–value pairs
- **Access:** Using keys
- **Add:** dict[new_key] = value
- **Update:** Change value using key

- **Delete:** del, pop()
- **keys():** Returns keys
- **values():** Returns values
- **items():** Returns key–value pairs

7. Working with Dictionaries

1. Iterating Over a Dictionary Using Loops

Definition

Iterating over a dictionary means accessing its keys, values, or key–value pairs one by one using loops (usually a for loop).

a) Iterating over keys

```
student = {"name": "Ali", "age": 20, "marks": 85}
```

```
for key in student:  
    print(key)
```

Output:

```
name  
age  
marks
```

b) Iterating over values

```
for value in student.values():  
    print(value)
```

Output:

```
Ali  
20  
85
```

c) Iterating over key–value pairs

```
for key, value in student.items():  
    print(key, ":", value)
```

Output:

```
name : Ali  
age : 20  
marks : 85
```

2. Merging Two Lists into a Dictionary

a) Using a Loop

Definition

Two lists (one for keys and one for values) can be merged into a dictionary by iterating over them using a loop.

Example

```
keys = ["name", "age", "marks"]  
values = ["Ali", 20, 85]
```

```
student = {}
```

```
for i in range(len(keys)):  
    student[keys[i]] = values[i]
```

```
print(student)
```

Output:

```
{'name': 'Ali', 'age': 20, 'marks': 85}
```

b) Using zip()

Definition

The zip() function combines elements from two lists into pairs, which can be directly converted into a dictionary.

Example

```
keys = ["name", "age", "marks"]  
values = ["Ali", 20, 85]
```

```
student = dict(zip(keys, values))  
print(student)
```

Output:

```
{'name': 'Ali', 'age': 20, 'marks': 85}
```

3. Counting Occurrences of Characters in a String Using Dictionaries

Definition

A dictionary can be used to count how many times each character appears in a string by storing characters as keys and their counts as values.

Example

```
text = "python"
count = {}

for char in text:
    if char in count:
        count[char] += 1
    else:
        count[char] = 1

print(count)
```

Output:

```
{'p': 1, 'y': 1, 't': 1, 'h': 1, 'o': 1, 'n': 1}
```

Example with repeated characters

```
text = "banana"
count = {}

for char in text:
    count[char] = count.get(char, 0) + 1

print(count)
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

- Dictionaries can be iterated using `keys()`, `values()`, and `items()`
- Two lists can be merged into a dictionary using loops or `zip()`
- Dictionaries are useful for counting occurrences of characters
- Keys store characters, values store counts

8. Functions

1. Defining Functions in Python

Definition

A **function** is a reusable block of code that performs a specific task. Functions help in **code reusability**, **better organization**, and **easy maintenance**.

Syntax

```
def function_name(parameters):  
    statements  
    return value
```

Example

```
def greet():  
    print("Hello, Python!")  
  
greet()
```

Output:

Hello, Python!

2. Types of Functions in Python

Functions can be classified based on **parameters** and **return values**.

a) Function Without Parameters and Without Return Value

Definition

This type of function does not take any input and does not return any output. It only performs a task.

Example

```
def message():  
    print("Welcome to Python")  
  
message()
```

b) Function With Parameters and Without Return Value

Definition

This function takes input values (parameters) but does not return a value.

Example

```
def add(a, b):  
    print(a + b)
```

```
add(5, 3)
```

Output:

8

c) Function Without Parameters and With Return Value

Definition

This function does not take any input but returns a value.

Example

```
def get_number():  
    return 10
```

```
x = get_number()  
print(x)
```

Output:

10

d) Function With Parameters and With Return Value

Definition

This function takes input values and returns an output value.

Example

```
def multiply(a, b):  
    return a * b
```

```
result = multiply(4, 5)  
print(result)
```

Output:

20

3. Anonymous Functions (Lambda Functions)

Definition

A **lambda function** is a small anonymous function defined using the lambda keyword. It can have **any number of arguments** but **only one expression**.

Syntax

lambda arguments : expression

Example

```
square = lambda x: x * x  
print(square(6))
```

Output:

36

Example with multiple arguments

```
add = lambda a, b: a + b  
print(add(3, 7))
```

Output:

10

- **Function:** Reusable block of code
- **def keyword:** Used to define a function
- **Parameters:** Inputs to a function
- **Return:** Sends value back to caller
- **Lambda function:** Anonymous, one-line function

9. Modules

1. Introduction to Python Modules and Importing Modules

Definition

A **module** in Python is a file that contains Python code such as functions, variables, and classes. Modules help in **organizing code**, **reusability**, and **maintainability**.

Importing Modules

Python provides different ways to import modules.

a) Importing the whole module

```
import math  
print(math.sqrt(25))
```

b) Importing a specific function

```
from math import sqrt  
print(sqrt(16))
```

c) Importing with an alias

```
import math as m  
print(m.factorial(5))
```

2. Standard Library Modules

a) math Module

Definition

The **math module** provides mathematical functions such as square root, power, factorial, etc.

Examples

```
import math  
  
print(math.sqrt(36))    # Square root  
print(math.pow(2, 3))   # Power  
print(math.factorial(5)) # Factorial
```

Output:

```
6.0  
8.0  
120
```

[EXTRA PRACTICE QUESTION\Demonstrate.py](#)

b) random Module

Definition

The **random module** is used to generate random numbers and perform random selections.

Examples

```
import random
```

```
print(random.randint(1, 10)) # Random integer
print(random.choice([10, 20, 30, 40])) # Random choice
```

3. Creating Custom Modules

Definition

A **custom module** is a user-defined Python file that contains functions or variables and can be imported into other programs.

Steps to Create a Custom Module

Step 1: Create a module file (mymodule.py)

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Step 2: Import and use the custom module

```
import mymodule

print(mymodule.add(5, 3))
print(mymodule.subtract(10, 4))
```

Output:

```
8
6
```

- **Module:** A file containing Python code
- **Import:** Used to access module content
- **math module:** Mathematical operations
- **random module:** Random number generation
- **Custom module:** User-defined Python file